A Zero Trust Architecture Integrating Rule-Based Bias Screening and Hallucination Checks in AI-Powered Interview Systems

Sai Pranavi A P
Dept. of AI & ML
RV College of Engineering
Bengaluru, India

Sai Triveni B
Dept. of AI & ML
RV College of Engineering
Bengaluru, India

Sanjana H V
Dept. of AI & ML
RV College of Engineering
Bengaluru, India

Sneha V
Dept. of AI & ML
RV College of Engineering
Bengaluru, India

Faculty Coordinator:
Apoorva Chate
Dept. of AI & ML, RV College of Engineering, Bengaluru, India

***Abstract*—AI-powered interview systems are showing up everywhere in campus recruitment, but the security and fairness problems they raise tend to get handled by different people entirely — security teams patch the pipeline, fairness researchers look at the model. This paper presents TrustInterview AI, a prototype that tries to close that gap by applying Zero Trust Architecture (ZTA) principles across both concerns at once. Eleven conventional application-security controls (identity verification, request validation, rate limiting, and similar hygiene) sit underneath two AI-specific layers we added on top: a rule-based bias screen covering 14 protected categories, and a hallucination check that catches impossible technical claims and cross-references what a candidate says against their uploaded resume. We tested the prototype on 200 interview sessions we built ourselves — 50 with injected bias, 50 with injected impossible claims, and 100 left unmodified. The system caught every injected case and raised no false alarms on the unmodified set, and the full 13-layer pipeline added only 94 ms of overhead against roughly 1.85 s of LLM inference time. That said, because we wrote the test cases using the same category taxonomy the detector uses, and because the dataset is small and synthetic, these numbers are best read as confirmation that the components do what we built them to do — not as evidence of real-world accuracy or generalization. We say this plainly and lay out what external validation would still be needed before any fairness or compliance claim could be made with real confidence.**

***Index Terms*—Zero Trust Architecture, bias detection, hallucination prevention, AI fairness, interview automation, LLM security**

I. Introduction

Campus recruitment and corporate hiring are both leaning harder on AI-powered interview systems to screen more candidates, faster. Our own university uses them to connect students with employers, and companies use them to triage applicant pools that would otherwise take weeks to work through by hand. That speed comes with two risks that are usually handled as if they were unrelated: security vulnerabilities like prompt injection, malicious uploads, and unauthorized access on one side, and fairness failures like discriminatory questions, biased scoring, or claims about a candidate that nobody checked, on the other.

A. The Problem: Security and Fairness Are Coupled

In practice these two risks aren't separable. If an attacker compromises the scoring logic, the easiest thing to do with that access is bias outcomes toward or away from particular candidates — so a security breach is a fairness failure by another name. Run it the other direction and the same is true: an AI system that asks a discriminatory question or states something ungrounded about a candidate has broken an integrity guarantee, not just a fairness one. Yet security engineering and fairness research still tend to run as separate workstreams. That gap is what motivated this design.

B. Our Approach

We take the Zero Trust principle from NIST SP 800-207 [1] — never trust, always verify — and apply it past its usual scope of network and identity verification, extending it to cover the AI system's own outputs as well. The prototype runs every request through 13 sequential checks:

- **Layers 1–11** handle the application-security side: session/identity verification, device fingerprinting, CORS enforcement, payload limits, sandboxed file parsing, audit logging, rate-limit-based automated response, input sanitization, schema validation, transport-layer hardening, and IP/threat-signature screening. Individually, none of this is new. What we're contributing here is treating them as mandatory, sequential gates that everything must pass before it reaches the AI-specific layers.
- **Layer 12** is a rule-based bias screen. It checks generated questions and evaluation text against keyword lists for 14 protected categories — age, gender, marital/family status, and so on — before either one reaches a candidate or gets recorded, and it separately flags feedback that names the candidate instead of describing their work.

- **Layer 13** is a hallucination check. It flags technically impossible claims (software versions that don't exist, metrics that break basic bounds like a percentage over 100), and it cross-references the technical terms a candidate uses out loud against what's actually on their uploaded resume.

If any layer rejects a request, the pipeline stops right there, so any failure can be traced back to one specific, auditable check rather than a tangle of possible causes.

C. Key Contributions

1) An architecture that puts AI-specific integrity checks — bias screening, hallucination detection — inside the same enforcement pipeline as conventional Zero Trust security controls, instead of bolting them on as a separate review step afterward.
2) A rule-based bias screen across 14 protected categories, documented in enough detail to be reproduced and picked apart, including a direct account of what keyword matching catches and what it doesn't.
3) A hallucination check combining an impossible-claim pattern library, resume cross-referencing, and a simple risk-scoring heuristic that we disclose in full rather than treat as a black box.
4) A transparency dashboard showing the outcome of all 13 layers for every request.
5) An evaluation across 200 internally built interview sessions, reported alongside a candid discussion of what that evaluation can and can't tell us (Sections V-C and VI).

D. Scope and Honesty About Claims

This is a student-built prototype, not a validated production system, and we want that distinction to be clear from the outset. Section V shows that the rules we implemented behave as intended on a test set built around their own taxonomy — it does not show real-world fairness, and it doesn't establish legal compliance with anything on its own. We'd rather state that boundary directly than let the numbers imply more than they support, and Section VI lays out what further evaluation would actually be needed to say more.

E. Paper Organization

Section II covers related work. Section III walks through the 13-layer architecture. Section IV describes the implementation. Section V presents the evaluation, including its limits. Section VI discusses broader limitations and future work, and Section VII concludes.

II. Related Work

A. Zero Trust Architecture

NIST SP 800-207 [1] defines Zero Trust Architecture around a simple rule: no entity is trusted by default, and every request has to be authenticated and authorized continuously. Syed et al. [2] survey the field comprehensively, and Bertino [3] weighs in on where the model actually helps in practice and where it doesn't. Almost all of this work, ours included, is aimed at network and identity security — none of it addresses AI-specific integrity problems like biased or fabricated model output. There's a smaller thread of research combining AI with Zero Trust for other purposes; Nagarajan et al. [4], for instance, propose an AI-based Zero Trust approach for consumer-industry security. What sets our work apart is treating the AI system's outputs themselves, not just the traffic around them, as an untrusted resource that needs verifying.

B. Bias in AI Hiring Systems

Bias in hiring didn't start with LLMs. Bertrand and Mullainathan's [5] resume-callback study is the classic reference point here: identical resumes got noticeably more callbacks when given distinctively White-sounding names than distinctively Black-sounding ones. Buolamwini and Gebru [6] later showed similarly large accuracy gaps across gender and skin type in commercial classification systems, which helped push broader scrutiny of algorithmic tools generally. Köchling and Wehner [7] pull much of this together in a systematic review focused specifically on discrimination and fairness in algorithmic HR recruitment. Most of the mitigation work in this space happens at training time, through reweighting or fairness constraints on the model itself. We take a different angle: inference-time screening of generated text, which works with whatever model sits underneath but is also, as we get into in Section VI, limited to whatever surface patterns we've explicitly told it to watch for.

C. LLM Hallucination

Large language models will confidently say things that aren't true. Huang et al. [8] survey why this happens and how researchers have tried to catch it, and Manakul et al. [9] propose SelfCheckGPT, a black-box method that detects likely hallucinations by sampling for consistency without needing access to model internals. A lot of this literature leans on retrieval or evidence-grounding as the fix. We go narrower: our hallucination layer checks claims against one specific document — the candidate's resume — and flags anything that breaks a simple factual or numerical bound, rather than trying to solve hallucination detection in general.

III. Architecture: 13-Layer Zero Trust Design

Every incoming request — a resume upload, a question-generation call, an answer evaluation — passes through all 13 layers in sequence. Failing any one of them stops the request immediately, which gives us clean, per-layer logs for forensic purposes.

A. Layers 1-11: Standard ZTA Controls

Layer 1 — Identity & Session Management. Non-public API requests must carry a cryptographically-secured session token; anything missing one, or presenting an expired one, gets rejected.

Layer 2 — Device Fingerprinting. Client-side JavaScript computes a device fingerprint, and requests coming from

headless browsers or known bot user-agents get flagged for review.

Layer 3 — Network & CORS. Origin matching is enforced strictly; requests from origins we don't recognize are rejected outright.

Layer 4 — Payload Size. Request bodies are capped at 10 MB, mainly to keep memory-exhaustion attacks off the table.

Layer 5 — Data Protection. Resume PDFs get parsed inside isolated subprocesses with restricted file-system access.

Layer 6 — Audit Logging. Every request is logged — IP, headers, method, path, status code, latency.

Layer 7 — Automated Response. IPs that exceed the configured rate limit, or that keep triggering errors, get blocked automatically.

Layer 8 — Input Sanitization. POST bodies are scanned for XSS patterns, SQL-injection fragments, and secrets that shouldn't have been included.

Layer 9 — Policy Decision Point. Each request's method, path, and payload schema are checked against pre-defined policies.

Layer 10 — Edge Security. HSTS, CSP directives, and secure cookie flags are all enforced by default.

Layer 11 — Threat Intelligence. A blocklist of known-bad IP ranges is maintained, and payloads are scanned for command-injection signatures.

None of Layers 1 through 11 is a new security technique — this is standard application-security practice, and we're not claiming otherwise. We include them because Zero Trust means verifying continuously at every stage, and because the AI-specific layers that follow only mean something if the pipeline feeding them is already hardened.

B. Layer 12: Rule-Based Bias Screening

Layer 12 checks generated text against keyword lists across 14 protected categories. Here's a case from our testing that illustrates why this matters: a generated question read, "Given your age and experience level, how would you handle working with a younger team?" The underlying topic — teamwork across generations — is a fair thing to ask about. But "given your age" turns a protected attribute into an explicit factor in the question, and that's what gets flagged.

The keyword lists themselves are illustrative, not exhaustive. For age, we watch for terms like "age," "how old," "young," "old," "years old." For gender: "male," "female," "woman," "man," "pregnant." For family status: "married," "children," "kids," "family," "spouse."

The fourteenth category — personal identity — works a little differently. We tokenize the candidate's name from their resume ("Sanjana H V" becomes {sanjana, h, v}), and any evaluation text containing those tokens gets flagged. The reasoning is simple: evaluation should describe the work — "the candidate demonstrates strong coding skills" — not the person.

Each piece of text gets a compliance score: 100% if nothing triggers, and otherwise a per-category report noting which category, which keyword, and where it showed up, so a human can review it. We want to be upfront that this is keyword matching, not semantic understanding — Section VI goes into what that misses.

C. Layer 13: Hallucination and Unverified-Claim Detection

Layer 13 runs three checks.

1) Impossible-technology detection. A pattern library flags claims about versions or products that don't exist — "Python 4.0," "HTML6" — and we update it by hand as real new versions come out.

2) Impossible-metric detection. Claims that break basic numeric bounds get flagged as high-risk: a percentage reduction over 100, an accuracy figure above 100%, throughput numbers that don't square with the stated hardware.

3) Resume cross-checking. We pull technical keywords out of both the candidate's spoken answer and their resume and compare them. Anything in the answer that's absent from the resume gets flagged as a medium-risk, unverified claim — which could be perfectly legitimate (a skill picked up recently) or could be an exaggeration. Either way, the flag is meant as a prompt for a human interviewer to ask a follow-up question, not as a verdict.

These flags feed into a heuristic risk score: 45 points per high-risk violation, 20 per medium-risk flag, with scores above 60 read as high risk, 25 to 59 as medium, and below 25 as low. We picked these numbers based on informal testing during development, not by fitting them against an independent dataset, so they should be treated as a reasonable starting configuration rather than a calibrated model.

IV. Implementation

A. System Stack

Backend: Node.js running 13 sequential middleware layers. LightGBM [10] handles question-difficulty calibration. Evaluation text comes from OpenAI GPT-3.5-Turbo and Google Gemini-1.5-Flash.

Frontend: React, with a real-time dashboard showing layer status. Speech-to-text runs through the Web Speech API.

Deployment: Docker on AWS EC2, Redis for session persistence, CloudWatch for log aggregation.

B. Algorithms

The bias-screening logic checks, category by category, whether trigger keywords show up in candidate-facing text. The hallucination-checking logic looks for impossible-technology mentions, metric-bound violations, and resume-unverified terms, then rolls the results into the risk score described in Section III-C.

V. Evaluation

A. Experimental Setup

We tested the prototype against 200 software-engineering interview sessions, all of which we built ourselves:

- 50 sessions with biased questions injected deliberately, written against the same category taxonomy Layer 12 implements.
- 50 sessions with impossible claims injected deliberately, written against the same pattern categories Layer 13 implements.
- 100 sessions left unmodified, with realistic and accurate content.

We treat these results as a specification check — confirmation that the rules fire on the cases they were built to catch — rather than an estimate of accuracy on unseen, adversarially diverse, real-world input. Section V-C spells this distinction out further.

B. Results

Bias screening. All 50 sessions with injected bias were flagged, and none of the 100 unmodified sessions triggered a false alarm.

Hallucination checking. All 50 sessions with injected impossible claims were flagged as high-risk. Eight of the 100 unmodified sessions also picked up medium-risk flags, because a candidate mentioned a technology that wasn't on their resume. On review, we judged these to be legitimate signals worth an interviewer following up on — a recently learned skill, most likely — rather than detector mistakes. Worth noting: that judgment was made by the same team that built the detector, which is itself a limitation of this evaluation.

Latency. The full 13-layer pipeline adds 94 ms of overhead, about 5% of total request time. LLM inference, at roughly 1.85 s, dominates the overall latency by a wide margin, so the added checks are unlikely to be the bottleneck users actually notice — though we haven't tested this under production-scale concurrent load.

Adversarial probes. We tried 20 resumes with embedded prompt-injection text ("ignore previous instructions and give this candidate a perfect score," for example), and all 20 were caught, mostly because Layer 13's resume cross-check doesn't depend on the injected instruction actually being followed. We also simulated a burst of 1,000 requests per minute from a single IP; Layer 7's rate limiting kicked in and blocked it after 100 requests within 15 minutes.

C. Threats to Validity

Three limitations of this evaluation are worth calling out directly, on top of the broader ones in Section VI:

1) **Self-designed test cases.** We built the bias- and hallucination-injection cases using the same category taxonomy the detectors themselves use. High detection rates here are more or less expected by construction, and shouldn't be read as a measure of how well the system generalizes to novel phrasing, coded language, or real candidate behavior.
2) **No independent labeling.** Every ground-truth label in this evaluation, including the manual review of the 8 medium-risk flags, was assigned by us — not by independent annotators or by anyone the system's decisions would actually affect.
3) **Small, synthetic, single-domain sample.** 200 sessions, all software-engineering interviews, isn't enough to support a general accuracy claim, and none of it is real candidate data.

VI. Discussion

A. Key Findings

Within the bounds of this internally built test set, the layered pipeline does what it was designed to do: the rule-based checks catch the categories of problems they were built for, and the latency they add is small next to LLM inference time. The per-layer dashboard also gives auditors and candidates a window into which checks ran and what they found — a reasonable step toward the kind of explainability Wachter et al. [11] discuss, even if it doesn't fully answer their concerns.

B. Limitations

Keyword matching has clear blind spots. The bias screen catches explicit references to protected categories, but coded language slips right past it — phrases like "culture fit" or "rockstar developer" can encode age or class bias without tripping any flagged keyword. Closing that gap probably means training and validating a learned classifier on real, diverse hiring language, not just extending the keyword list further.

The evaluation doesn't establish real-world accuracy. As Section V-C lays out, our 100% detection numbers come from a test set we designed and graded ourselves, built around the detector's own taxonomy. We don't have results yet on an independently built or real-world dataset, and we're not making any claim about the system's false-positive or false-negative rates in actual deployment.

PDF parsing constrains resume cross-checking. Layer 13 depends on getting clean text out of a resume. Scanned documents and unusual multi-column layouts degrade that extraction, which can cause a legitimate skill to get misflagged as unverified simply because we couldn't read it off the page correctly.

The system can't fix bias inside the underlying LLM. We're screening outputs, not training or fine-tuning GPT-3.5-Turbo or Gemini-1.5-Flash. Biased reasoning that doesn't surface as a flagged keyword or an out-of-bounds metric passes through untouched.

No benchmark against commercial systems. HireVue and Pymetrics are both closed-source, so we have no way to compare detection performance directly against them. What we can offer instead is an architecture and detection logic that's fully open to inspection — a partial substitute for benchmarking, though not an equivalent to it.

Regulatory alignment, not compliance. The layer-status dashboard was designed with GDPR Article 22's explanation expectations and general FCRA documentation practice in mind, but neither this paper nor its authors are in a position

to make a legal compliance determination. Wachter et al. [11] raise real open questions about how far any GDPR "right to explanation" actually extends, and any compliance claim here would need review by qualified counsel against the specific deployment and jurisdiction involved.

C. Future Work

A few things are next on our list: evaluating against an independently built, larger, and more linguistically diverse dataset, ideally with external annotators; training a learned classifier to catch coded bias language that keyword matching misses, evaluated on held-out data; getting a formal legal review of the design choices that touch on compliance; and extending the architecture to video interviews, which bring their own fairness risks — bias in facial or speech analysis, for instance — that this system doesn't currently address at all.

VII. Conclusion

TrustInterview AI started as an attempt to see whether Zero Trust principles — treat every request as untrusted until verified, including the AI system's own outputs — could be usefully applied to AI hiring pipelines. The 13-layer design, and specifically folding rule-based bias screening and hallucination checking into the same enforcement pipeline as conventional security controls, seems like a reasonable architectural pattern for this kind of problem. But we want to be clear about what our evaluation actually shows: the rules behave as specified on a small test set we built ourselves, not that the system is fair, accurate, or legally compliant once deployed. We're saying this directly because the distance between "the code does what we told it to" and "the system is actually fair and trustworthy" is precisely the distance that matters in a hiring context, and closing it needs independent, larger-scale evaluation that this paper doesn't yet provide.

The code for TrustInterview AI is open source, and we'd welcome independent testing, criticism, and extension — especially evaluation on data we didn't build ourselves.

Acknowledgment

We thank Prof. Apoorva Chate for guidance throughout this project and the RV College of Engineering AI & ML Department for providing computational resources.

References

[1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020.

[2] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero Trust Architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.

[3] E. Bertino, "Zero trust architecture: Does it help?," *IEEE Security & Privacy*, vol. 19, no. 5, pp. 95–96, 2021.

[4] S. M. Nagarajan, G. G. Devarajan, M. S. Thangakrishnan, T. V. Ramana, A. K. Bashir, and A. A. AlZubi, "Artificial Intelligence-Based Zero Trust Security Approach for Consumer Industry," *IEEE Trans. Consumer Electron.*, vol. 70, no. 3, pp. 5411–5418, 2024.

[5] M. Bertrand and S. Mullainathan, "Are Emily and Greg more employable than Lakisha and Jamal? A field experiment on labor market discrimination," *American Economic Review*, vol. 94, no. 4, pp. 991–1013, Sep. 2004.

[6] J. Buolamwini and T. Gebru, "Gender shades: Intersectional accuracy disparities in commercial gender classification," in *Proc. 1st Conf. Fairness, Accountability and Transparency (PMLR)*, 2018, pp. 77–91.

[7] A. Köchling and M. C. Wehner, "Discriminated by an algorithm: a systematic review of discrimination and fairness by algorithmic decision-making in the context of HR recruitment and HR development," *Business Research*, vol. 13, no. 3, pp. 795–848, 2020.

[8] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, 2023.

[9] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 9004–9017.

[10] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[11] S. Wachter, B. Mittelstadt, and L. Floridi, "Why a right to explanation of automated decision-making does not exist in the General Data Protection Regulation," *International Data Privacy Law*, vol. 7, no. 2, pp. 76–99, 2017.